

SEA Food IP_14

Final Project Technical Report

SE/COM S 3190 – Construction of User Interfaces Spring 2025

Team Members:

Member 1 - trungly@iastate.edu

Member 2 - dcavan@iastate.edu

May 11, 2025

1. Introduction

Overview of the problem, motivation, users, and goals. State whether the project is original or inspired.

We noticed that most food places do not have websites for their restaurants. Especially for local businesses. We believe that having a functional restaurant website can help increase revenue and make even profit from better marketing. Although you can use apps like doordash, uber eats, for catering, the problem is that it is generally more expensive due to fees.

This project is inspired from looking at other well made restaurants websites from large cities like New York, Los Angeles, etc. We take a deep dive into studying how they present their website, and what products they offer.

2. Project Description

Explain major features, user flow, CRUD operations and entities affected.

Menus Pages - Users can browse through the menu page including entrees, drinks, and desserts

Shopping Cart - Users can add items to their cart, look at the total price and adjust quantities or remove items from the cart.

Order Placement - Users fill out the payment information to place the order which sends that submitted data to the backend

Order Summary - Upon successful submission, users are shown a summary of their order.

Contact Form - Users can submit their message through contact form, which passes to the backend to store that information

1. User opens the homepage.
2. User selects a category from the menu (entrees, drinks, or desserts).
3. Users add items to their cart.
4. User proceeds to the checkout page.
5. User fills in contact/payment information and submits the order.
6. Order is saved, and the user is shown a summary.

Operation	Endpoint Example	Description	Entity
Create	POST /cart	Adds items to the cart	cart
Read	GET /products, GET /drinks, GET /desserts	Fetches menu items	products, drinks, desserts
Read	GET /cart	Fetches current cart contents	cart
Update	PUT /cart/:product_id	Updates quantity of a cart item	cart
Delete	DELETE /cart/:product_id	Removes an item from the cart	cart
Create	POST /orders	Submits a new order	orders
Create	POST /inquiries	Submits a contact form message	inquiries

3. File and Folder Architecture

Describe structure of frontend/, backend/, Documents/. Include a folder tree diagram.

IP_14/

- |— README.md
- |— package.json
- |— Documents/
 - | — Mini-Assingnment 1/
- |— src/
 - | |— Documents/
 - | | |— Final project proposal.pdf
 - | | | — IP14_MT_Proposal (4).pdf
 - | |— backend/
 - | | |— assets/
 - | | | |— data.json
 - | | |— insertAlcohol.js
 - | | |— insertDessertsData.js
 - | | |— insertDrinks.js
 - | | |— insertEntrees.js
 - | | | — server.js
 - | | — frontend/
 - | | | — final_app/
 - | | | |— index.html
 - | | | |— package.json
 - | | | |— tailwind.config.js
 - | | | |— vite.config.js
 - | | | |— src/
 - | | | | |— App.jsx
 - | | | | |— About_Us.jsx

- | | └─ Contact.jsx
- | | └─ EntreeMenu.jsx
- | | └─ OrderSummary.jsx
- | | └─ PaymentInfo.jsx
- | | └─ ViewCart.jsx
- | | └─ ViewMenu.jsx
- | | └─ dessertsMenu.jsx
- | | └─ drinksMenu.jsx
- | | └─ assets/
- | | └─ images/
- | | └─ index.css
- | | └─ main.jsx

4. Code Explanation and Logic Flow

4.1. Frontend–Backend Communication

Describe how API requests are made and handled.

GET - fetch the products for the items

POST - Add new orders or messages

PUT - update an item quantities on cart

DELETE - Remove items from the cart

Routes are defined using Express and are mapped to URL paths like /cart, /products,

These requests get parsed, perform DB operations (Insert, Update, Delete) and return JSON responses.

4.2. React Component Structure

Overview of component hierarchy, use of props/state.

The frontend is structured around the main component, App.jsx that manages the navigation and React's use state hook globally. It is split into multiple page components and menu components, which is conditionally rendered on the current step.

App.jsx - The root component

Menus (EntreeMenu.jsx , drinksMenu.jsx , dessertsMenu.jsx) - Render lists of items with Add To Cart functionality.

ViewCart.jsx: Displays cart contents, lets users update quantity or remove items.

PaymentInfo.jsx: Collects user contact and payment details.

OrderSummary.jsx: Shows the final order after submission.

Contact.jsx: Provides a contact form for user messages.

About_Us.jsx: Displays team/project info.

```
const [items, setItems] = useState([]);

// Fetch all cart items from backend
useEffect(() => {
  fetch("http://localhost:3000/cart")
    .then((res) => res.json())
    .then((data) => {
      setItems(data);
    })
    .catch((err) => console.error("Error fetching cart items:", err));
}, []);
```

useState : Steps to control which page is currently visible,

setForms to render the information being filled on the frontend and process that to the backend.

Cart for storing the array on the backend.

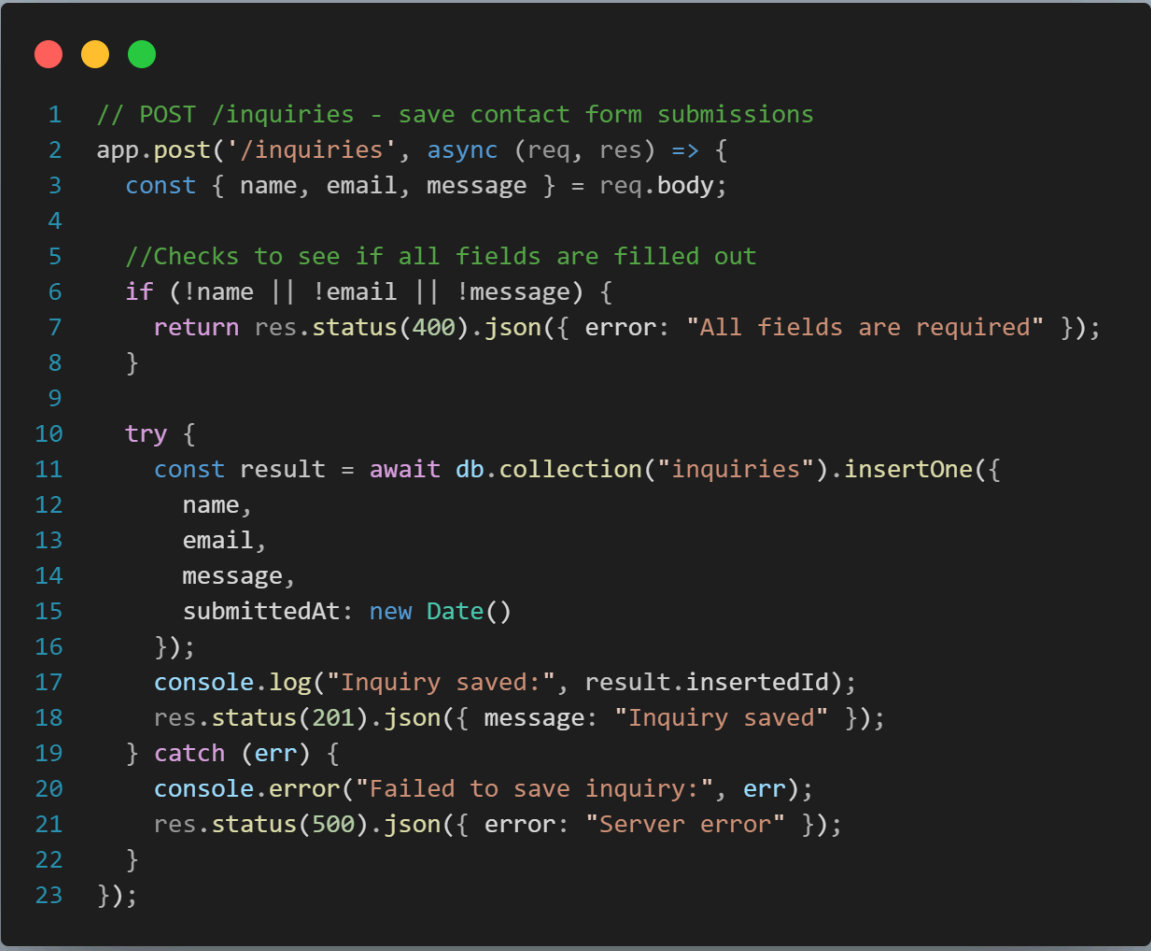
items for storing the fetched lists from the backend.

useEffect for CRUD operations when the user clicks on remove item/ add item on the cart or add to cart.

4.3. Database Interaction

The data base is used to store our menu items, such as Drinks deserts and alcohol. It is also used to store inquiries from the contact us page.

4.4. Code Snippets



```
1 // POST /inquiries - save contact form submissions
2 app.post('/inquiries', async (req, res) => {
3   const { name, email, message } = req.body;
4
5   //Checks to see if all fields are filled out
6   if (!name || !email || !message) {
7     return res.status(400).json({ error: "All fields are required" });
8   }
9
10  try {
11    const result = await db.collection("inquiries").insertOne({
12      name,
13      email,
14      message,
15      submittedAt: new Date()
16    });
17    console.log("Inquiry saved:", result.insertedId);
18    res.status(201).json({ message: "Inquiry saved" });
19  } catch (err) {
20    console.error("Failed to save inquiry:", err);
21    res.status(500).json({ error: "Server error" });
22  }
23 });
```

This is a Post from the inquiries section, it will first check to make sure that the boxes have some data in it then it will try and save it to the data base.

5. Web View Screenshots and Annotations

(David)

SEA FOOD

HomeMenuCartContact usAbout Us

Contact Us

Name

Email

Message

Send Message

We would love to hear from you! Whether it's feedback, quick hello or looking for a job. Drop us a message and we'll respond as soon as possible.

Address : 123 SEAFOOD Street, Flavortown, SEA 10010
Phone # : (123) 456-7890
Email : hello@seafood.com

Contact us page allows users to submit a message to us saving the data given to MongoDB. We will record their name, Email, and the message.
(David)

SEA FOOD

HomeMenuCartContact usAbout Us

Payment Information

Full Name

Card Number

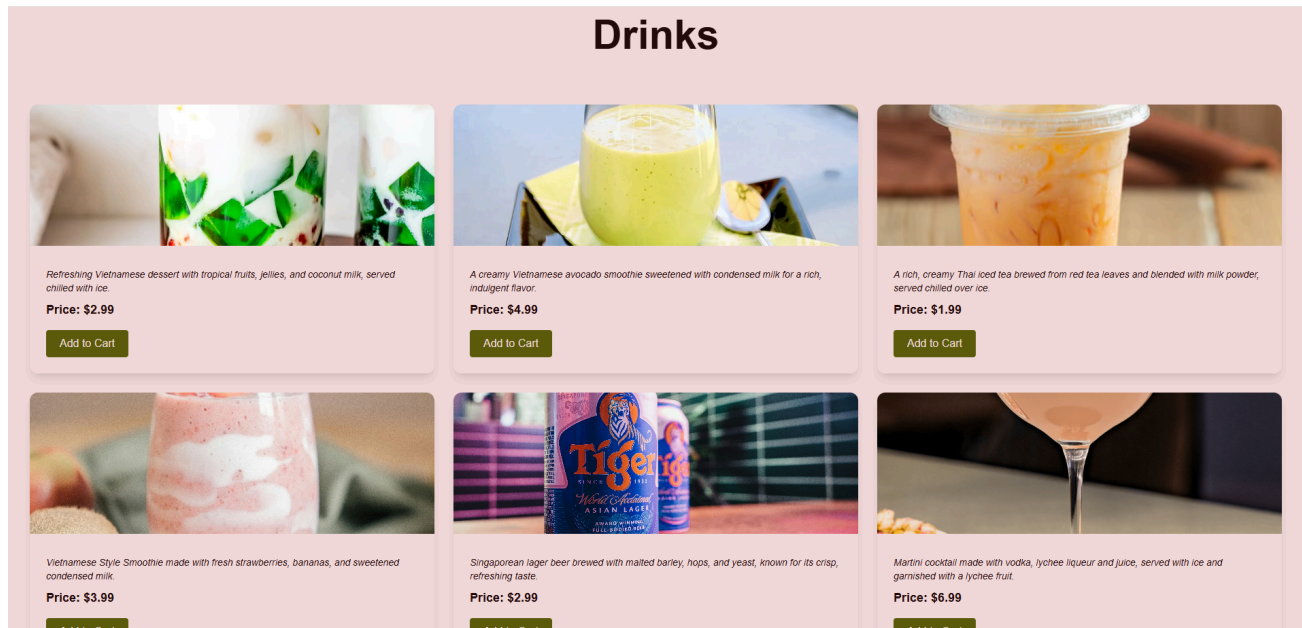
Delivery Address

Subtotal: \$30.96
Fees: \$5.00
Taxes (7%): \$2.17
Total: \$38.13

Submit Payment

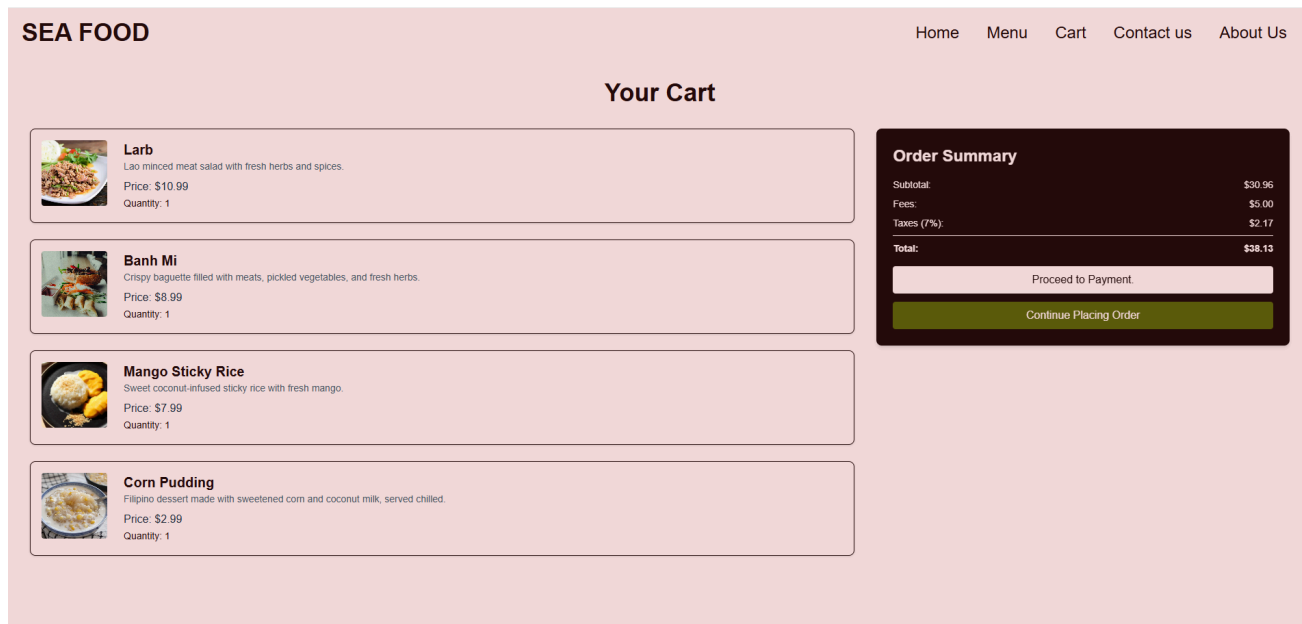
Payment Info, this allows user to input their credit card information to be charged for the food that was ordered.

(Trung)



Drinks menu, this menu allows users to view the available drinks and then add them to the cart for later checking out.

(Trung)



Cart Page, This allows user to view the items that they are wanting to purchase and proceed to pay for the items or add more.

6. Installation and Setup Instructions

Step-by-step instructions to run frontend and backend. Mention environment variables and DB setup.

1. `cd \src\backend`

2. src start (This will start up [server.js](#) and connect the data to the website)
3. open new terminal
4. cd\src \frontend\final_app
5. npm run dev
6. open new terminal

7. Contribution Overview

DessertsMenu	Trung
DrinksMenu	Trung
EntreeMenu	Trung
ViewMenu	Trung
ViewCart	Trung
Contact	David
OrderSummary	David
PaymentInfo	David
About Us	David

8. Challenges Faced

List 2–3 significant development/debugging issues and how you addressed them.

One of the big challenges we faced when developing this website was getting the frontend to properly communicate with the backend server. How we addressed this was, we made sure that the URL was correct and that it had the correct headers and even after all of that it was hard to test it since we had a hard time indicating to the user if a request succeeded or failed. Therefore we added confirmation messages to the UI so that the user is able to see that their request went through.

Another big challenge was running the localhost whenever there is a change. When you run `npm run dev`, it will give an error due to node modules not installed. This caused an interruption in our workflow and we worked together to fix the issue. We solved it by removing the node modules and reinstalling them from the root.

9. Final Reflections

working on this project gave us an opportunity to work on both frontend and backend developing this project from scratch. We learned how to structure a React application, Integrate MongoDB to store and retrieve data, collaborate effectively with another developer, etc. If we had more time we would have added User account creation so that we can provide deals to our customers and have a user order history. Overall this project pushed us to think like real developers where we experienced working as a full stack team